

LAB MANUAL ON CYBER SECURITY ESSENTIALS (R2024)

B.TECH III YEAR – V SEM (R24)

(2026-2027)

DEPARTMENT OF INFORMATION TECHNOLOGY

WEEK - 1

Writing simple Python scripts for tasks like string manipulation, reading from and writing to files, basic network communication.

WEEK - 2

Implementing basic encryption and decryption algorithms in Python Caesar cipher, AES, DES

WEEK - 3

Using python to generate and verify hashes (MD5, SHA-256) for files and messages.

WEEK - 4

Building a simple Python Client-Server application, understanding sockets.

WEEK - 5

Writing a python script to capture and analyze network packets(using libraries like Scapy or PySpark

WEEK - 6

Using python to interact with security-related APIs (eg. VirusTotal, Shodan)

WEEK - 7

Writing python scripts for basic static malware analysis (file signature analysis, string extraction).

AIM: To perform string manipulation, reading from and writing to files, and basic network communication using python scripts.

ALGORITHM:

1. Start
2. Define a function reverse_string(s)
 - Accept a string s
 - Return the reversed string using slicing (s[::-1])
3. Define a function read_and_reverse_write(input_file, output_file)
 - Open input_file in read mode
 - Read its contents
 - Reverse the content using reverse_string()
 - Open output_file in write mode
 - Write the reversed content into it
 - Close both files
4. Define a function simple_server(port)
 - Create a TCP socket
 - Bind the socket to localhost and the given port
 - Start listening for incoming connections
 - Accept a client connection
 - Receive data from the client
 - If data exists, send the same data back (echo)
 - Continue until no data is received
 - Close the connection
5. Main Execution
 - Create a string: "Hello, World!"
 - Reverse the string using reverse_string()
 - Display original and reversed strings
 - Call read_and_reverse_write('input.txt', 'output.txt')
 - (Optional) Run simple_server(65432) if network communication is needed
6. Stop

Python Code:

```
import socket

# String Manipulation Function
def reverse_string(s):
    return s[::-1]

# File Read and Write Function
def read_and_reverse_write(input_file, output_file):
    with open(input_file, 'r') as file:
        content = file.read()
        reversed_content = reverse_string(content)
    with open(output_file, 'w') as file:
        file.write(reversed_content)

# Basic Network Communication Function
def simple_server(port):
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.bind(('localhost', port))
    server_socket.listen(1)
    print(f"Server listening on port {port}...")
    conn, addr = server_socket.accept()
    print(f"Connected by {addr}")
    while True:
        data = conn.recv(1024)
        if not data:
            break
        conn.sendall(data)
    conn.close()

# Main Execution
if __name__ == "__main__":
    # String Manipulation
    original_string = "Hello, World!"
    reversed_str = reverse_string(original_string)
    print(f"Original String: {original_string}")
    print(f"Reversed String: {reversed_str}")
```

```
# File Read and Write
```

```
read_and_reverse_write('input.txt', 'output.txt')
```

```
# Simple Network Communication (Uncomment to run the server)
```

```
# Note: Running the server will require a client to connect and send data. #
```

```
simple_server(65432)
```

RESULT:

Thus the above program was executed successfully and got the desired output.

OUTPUT:

AIM: To implement basic encryption and decryption algorithms using python script.

ALGORITHM:

Caesar Cipher Algorithm (Substitution Cipher)

Encryption Algorithm

1. Start
2. Input plaintext and key (shift value)
3. For each character in plaintext:
 - Convert character to its alphabet position
 - Shift by key value
 - Wrap around if it exceeds alphabet range
4. Generate ciphertext
5. Display ciphertext
6. Stop

Decryption Algorithm

1. Start
2. Input ciphertext and key
3. For each character:
 - Convert to alphabet position
 - Subtract key
 - Apply modulo 26
4. Generate plaintext
5. Display plaintext
6. Stop

AES (Advanced Encryption Standard) Algorithm

AES is a symmetric block cipher.

Encryption Algorithm

1. Start
2. Input plaintext and secret key
3. Generate round keys using key expansion
4. Perform initial AddRoundKey
5. Repeat rounds:
 - SubBytes
 - ShiftRows
 - MixColumns
 - AddRoundKey
6. Perform final round (without MixColumns)
7. Generate ciphertext
8. Stop

Decryption Algorithm

- 1. Start**
 - 2. Input ciphertext and secret key**
 - 3. Generate round keys**
 - 4. Perform inverse operations:**
 - **InvShiftRows**
 - **InvSubBytes**
 - **AddRoundKey**
 - **InvMixColumns**
 - 5. Recover plaintext**
 - 6. Stop**
-

3. DES (Data Encryption Standard) Algorithm

DES is a symmetric block cipher using 56-bit key.

Encryption Algorithm

- 1. Start**
- 2. Input plaintext and key**
- 3. Apply Initial Permutation (IP)**
- 4. Split into Left (L) and Right (R) blocks**
- 5. Perform 16 rounds:**
 - **Expansion permutation**
 - **XOR with round key**
 - **S-box substitution**
 - **Permutation**
 - **Swap blocks**
- 6. Combine blocks**
- 7. Apply Final Permutation (FP)**
- 8. Generate ciphertext**
- 9. Stop**

Decryption Algorithm

- 1. Start**
- 2. Input ciphertext and key**
- 3. Apply Initial Permutation**
- 4. Perform same 16 rounds in reverse key order**
- 5. Apply Final Permutation**
- 6. Recover plaintext**
- 7. Stop**

Python Code:

```
from Crypto.Cipher import AES, DES
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad
import base64

# Caesar Cipher
def caesar_cipher_encrypt(text, shift):
    result = ""
    for i in range(len(text)):
        char = text[i]
        if char.isupper():
            result += chr((ord(char) + shift - 65) % 26 +
65) else:
            result += chr((ord(char) + shift - 97) % 26 +
97) return result

def caesar_cipher_decrypt(text, shift):
    return caesar_cipher_encrypt(text, -shift)

# AES Encryption/Decryption
def aes_encrypt(plain_text, key):
    cipher = AES.new(key, AES.MODE_CBC)
    ct_bytes = cipher.encrypt(pad(plain_text.encode('utf-8'), AES.block_size))
    iv = base64.b64encode(cipher.iv).decode('utf-8')
    ct = base64.b64encode(ct_bytes).decode('utf-8')
    return iv, ct

def aes_decrypt(iv, ct, key):
    iv = base64.b64decode(iv)
    ct = base64.b64decode(ct)
    cipher = AES.new(key, AES.MODE_CBC, iv)
    pt = unpad(cipher.decrypt(ct), AES.block_size)
    return pt.decode('utf-8')

# DES Encryption/Decryption
def des_encrypt(plain_text, key):
    cipher = DES.new(key, DES.MODE_CBC)
    ct_bytes = cipher.encrypt(pad(plain_text.encode('utf-8'), DES.block_size))
```

```

iv = base64.b64encode(cipher.iv).decode('utf-8')
ct = base64.b64encode(ct_bytes).decode('utf-8')
return iv, ct

def des_decrypt(iv, ct, key):
    iv = base64.b64decode(iv)
    ct = base64.b64decode(ct)
    cipher = DES.new(key, DES.MODE_CBC, iv)
    pt = unpad(cipher.decrypt(ct), DES.block_size)
    return pt.decode('utf-8')

# Main Execution
if __name__ == "__main__":
    # Caesar Cipher
    Example shift = 4
    original_text = "HelloWorld"
    encrypted = caesar_cipher_encrypt(original_text, shift)
    decrypted = caesar_cipher_decrypt(encrypted, shift)
    print(f"Caesar Cipher: {original_text} -> {encrypted} -> {decrypted}")

# AES Example
aes_key = get_random_bytes(16) # AES key must be either 16, 24, or 32 bytes long iv,
encrypted = aes_encrypt(original_text, aes_key)
decrypted = aes_decrypt(iv, encrypted, aes_key)
print(f"AES: {original_text} -> {encrypted} -> {decrypted}")

# DES Example
des_key = get_random_bytes(8) # DES key must be 8 bytes long iv,
encrypted = des_encrypt(original_text, des_key)
decrypted = des_decrypt(iv, encrypted, des_key)
print(f"DES: {original_text} -> {encrypted} -> {decrypted}")

```

RESULT:

Thus the above program was executed successfully and got the desired output.

EX:3

AIM: To generate and verify hashes for files and messages, utilizing hashing algorithms like MD5 and SHA-256 using python scripts.

ALGORITHM:

Algorithm:

1. Start.
2. Read input message M.
3. Convert message into binary.
4. Add padding bits to make length fit MD5 blocks.
5. Append original message length.
6. Divide into 512-bit blocks.
7. Initialize MD5 buffer values.
8. Perform MD5 compression rounds.
9. Combine final values.
10. Generate 128-bit hash H.
11. Display hash.
12. Stop.

Verification Algorithm (MD5):

1. Start.
2. Read original message and stored MD5 hash.
3. Generate MD5 hash of received message.
4. Compare generated hash with stored hash.
5. If equal → Data Verified.
6. Else → Data Modified / Verification Failed.
7. Stop.

2. SHA-256 Hash Generation Algorithm

Input: Message/Data M

Output: SHA-256 Hash H

Algorithm:

1. Start.
2. Read input message M.
3. Convert message to binary.
4. Add padding bits.
5. Append message length.
6. Divide into 512-bit blocks.

7. **Initialize SHA-256 hash values.**
8. **Execute 64 rounds of compression.**
9. **Merge final hash values.**
10. **Produce 256-bit hash H.**
11. **Display hash.**
12. **Stop.**

Verification Algorithm (SHA-256):

1. **Start.**
2. **Read original message and stored SHA-256 hash.**
3. **Generate SHA-256 hash of received message.**
4. **Compare both hashes.**
5. **If equal → Verification Successful.**
6. **Else → Integrity Check Failed.**
7. **Stop.**

Python Code:

```
import hashlib

# Function to generate MD5 hash
def generate_md5_hash(input_string):
    md5_hash = hashlib.md5()
    md5_hash.update(input_string.encode())
    return md5_hash.hexdigest()

# Function to generate SHA-256 hash
def generate_sha256_hash(input_string):
    sha256_hash = hashlib.sha256()
    sha256_hash.update(input_string.encode())
    return sha256_hash.hexdigest()

# Function to verify a hash
def verify_hash(input_string, known_hash):
    # Generate hash for the input string
    generated_hash = generate_md5_hash(input_string) if len(known_hash) == 32 else
generate_sha256_hash(input_string)

    # Compare the generated hash with the known hash return
    generated_hash == known_hash

# Main Execution
if __name__ == "__main__":
    # Example strings
    example_string1 = "HelloWorld"
```

```
example_string2 = "HelloWorld!"

# Generate hashes
md5_hash_example1 = generate_md5_hash(example_string1)
sha256_hash_example1 = generate_sha256_hash(example_string1)

print(f"MD5 Hash of '{example_string1}': {md5_hash_example1}")
print(f"SHA-256 Hash of '{example_string1}':
{sha256_hash_example1}")

# Verifying hashes
print(f"Verifying MD5 Hash of '{example_string1}': {verify_hash(example_string1, md5_hash_example1)}")
print(f"Verifying SHA-256 Hash of '{example_string1}': {verify_hash(example_string1,
sha256_hash_example1)}")

# Verifying incorrect hash
print(f"Verifying SHA-256 Hash of '{example_string2}' with hash of '{example_string1}':
{verify_hash(example_string2, sha256_hash_example1)}")
```

Result:

Thus the above program generate hashes and it hash value are also verified and the program executed successfully.

AIM: To build a simple python client-server application to understand the basics of socket programming.

ALGORITHM:

Server Algorithm

1. Start
2. Import the socket library.
3. Create a server socket using TCP protocol.
4. Bind the server socket to an IP address and port number.
5. Put the server into listening mode.
6. Wait for a client connection request.
7. Accept the client connection.
8. Receive data/message from the client.
9. Display the received message.
10. Send a response back to the client.
11. Close the client connection.
12. Close the server socket.
13. Stop

Client Algorithm

1. Start
2. Import the socket library.
3. Create a client socket using TCP protocol.
4. Connect to the server using IP address and port number.
5. Enter and send a message to the server.
6. Receive response from the server.

7. Display the received response.

8. Close the client socket.

9. Stop

Python Codes:

server.py (Server Script)

```
import socket

def start_server(host='localhost', port=65432):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.bind((host, port))
        s.listen()
        print(f"Server listening on {host}:{port}")
        conn, addr = s.accept()
        with conn:
            print(f"Connected by {addr}")
            while True:
                data = conn.recv(1024)
                if not data:
                    break
                conn.sendall(data)

if __name__ == "__main__":
    start_server()
```

Client.py (Client Script)

```
import socket

def start_client(host='localhost', port=65432):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((host, port))
        s.sendall(b'Hello, server')
        data = s.recv(1024)
        print(f"Received {data.decode()}")

if __name__ == "__main__":
    start_client()
```

RESULT:

Thus the above client -server program was executed successfully and got the desired output.

1.